

Lab 5: Parallel Programming

1. Implement a parallel program that counts how many odd numbers exist in a range (e.g., 1 to 10 million). Each thread should handle a subrange and maintain a local counter. The final count is the sum of all thread-local counters.
2. Implement a parallel histogram calculator.
 - Given an array of integers (e.g., pixel values from 0-255), each thread counts the frequency of numbers in its chunk.
 - Combine the frequency counts at the end to produce the final histogram.
3. Implement a parallel program to search for a target string in a large array of strings. Each thread searches a chunk of the array. If one thread finds the string, it reports the index and stops all other threads. Use a shared flag to synchronize early stopping.
4. Implement a parallel matrix multiplication program using threads. Given two $N \times N$ matrices A and B, write a C program that computes matrix $C = A * B$ in parallel. Each thread should handle one row of the resulting matrix. Ensure proper synchronization and memory management.
5. Implement a parallel quicksort algorithm using threads.
 - Given an array of integers, recursively partition the array around a pivot.
 - Spawn new threads to sort the left and right partitions concurrently.
 - For small partitions ($\leq$ THRESHOLD), use sequential quicksort.